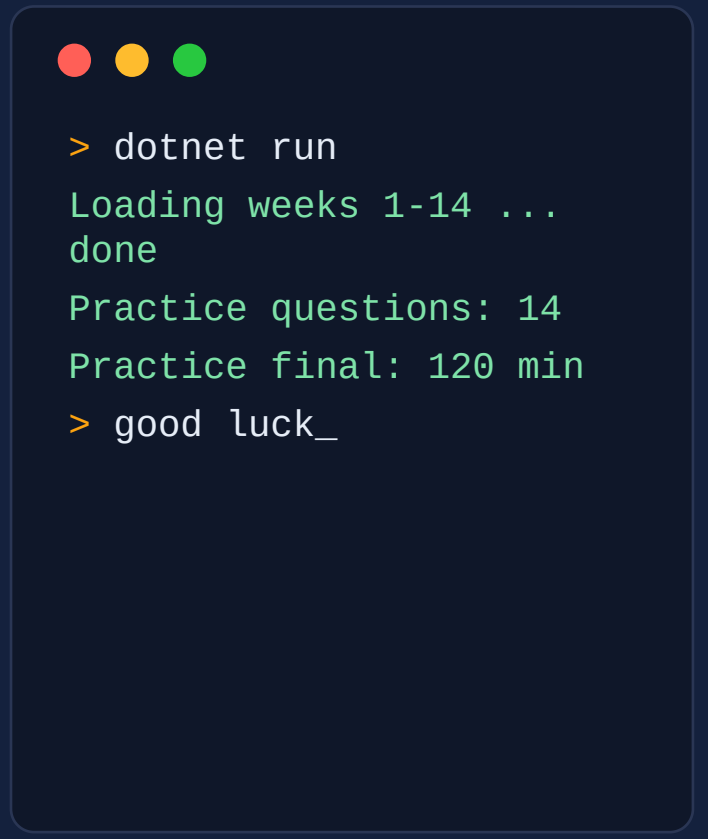
>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 15

Final Exam Review & Practice Final

Weeks 1–14 condensed · focus on arrays, matrices, methods, files · 14 practice questions



Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

>_ TODAY

What We Will Do in This Session

Review

Weeks 1–7 in one slide, then one slide per topic of Weeks 9–14.

Practise

14 exam-style questions: trace sorts and searches, matrices, calls, recursion, scope, bugs, files.

Plan

A 120-minute time plan and a method for each question type.

Self-test

Take the practice final in the notes under exam conditions, then check it.

Session timeline (approx.)

Logistics 10 min	Part 1: review 30 min	Parts 2–3: practice questions 60 min	Strategy 10 min

Rule for today: answer on paper BEFORE the answer slide. The exam measures what you can produce, not what you recognise.

>_ EXAM LOGISTICS

The Final Exam at a Glance

When & where

Date, room and duration: announced on the course group.

Materials

Expected closed book, as in the midterm. Confirmed in class.

Bring

Student ID card, pen. Pencil and eraser for diagrams and traces.

Weight

Final 60 % · Midterm 40 % of the course grade.

Calculators

Not needed for any question type. Policy: announced in class.

Scope

Weeks 1–14, cumulative. Focus on Weeks 9–14.

>_

Announcements

Official date, room, duration and exam rules are announced in class and on the course page.

What the Final Covers: Weeks 1–14

Week	Topic	Typical final-exam task	
1–2	Algorithms, flowcharts	pseudocode, flowchart of a loop with a decision	basis
3–4	Types, variables, operators	evaluate expressions; integer division; casts; Parse	basis
5–7	if/switch, for, while, do-while	inside every program; validation loops	basis
9	1-D arrays	indices, Length, traversal, reference vs copy	**FOCUS**
10	Searching, sorting, max/min	trace linear/binary search, bubble/selection passes	**FOCUS**
11	Matrices (2-D arrays)	nested loops, row/column sums, diagonals, output	**FOCUS**
12	Methods, parameters	write methods; value vs ref vs array parameters	**FOCUS**
13	return, void, scope, recursion	every path returns; scope errors; recursion trace	**FOCUS**
14	Files, try/catch	ReadAllLines/StreamReader, Split CSV, exceptions	**FOCUS**



Still not allowed in answers: LINQ (.Max(), .Sum() …) and List<T>. Use arrays and loops.

The Semester in One Picture

DATA

variable	1-D array	matrix	file
`int x`	`int[] a`	`int[,] m`	`grades.csv`
W3	W9	W11	W14

CONTROL

3 structures	algorithms	methods	return · scope
`if · for · while`	`search · sort`	`static int F(…)`	`recursion`
W1–7	W10	W12	W13



A typical final-exam program: read a file into an array → process it with small methods (search, sort, count) → write or print the result.



PART 1

Condensed Review

Weeks 1–7 in one slide, then arrays, search & sort, matrices, methods, scope, recursion, files

The First Half: Traps That Still Cost Points

Expression / code	Result	Rule
`7 / 2  7 / 2.0`	`3  3.5`	int / int is integer division
`-17 % 5`	`-2`	% keeps the sign of the left operand
`"Sum: " + 1 + 2`	`Sum: 12`	string + number, left to right
`int y = x++ * 2;  (x = 5)`	`y = 10, x = 6`	postfix: use, then increase
`b != 0 && a / b > 2  (b = 0)`	`false`	short-circuit: no division
`int.TryParse("12a", out int v)`	`false, v = 0`	no exception (Parse would throw)
`(int)7.9`	`7`	a cast truncates

Choose the loop

`for`

count known (arrays!)

`while`

condition, sentinel, ReadLine() != null

`do-while`

at least once: menus, validation

`foreach`

read every element, no index

One-Dimensional Arrays: Boxes With Indices

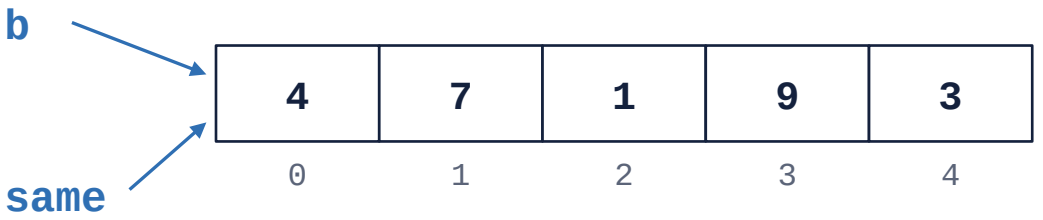
Arrays in 17 lines

```
int[] a = new int[5];          // 0 0 0 0 0
int[] b = { 4, 7, 1, 9, 3 };   // initializer
b[0] = 10;                     // write
int last = b[b.Length - 1];    // read: 3

for (int i = 0; i < b.Length; i++)
{
    a[i] = 2 * b[i];           // index needed
}
foreach (int v in b)           // read-only
{
    Console.Write(v + " ");
}
Console.WriteLine(string.Join(", ", a));

int[] same = b;                // same array!
int[] copy = (int[])b.Clone(); // new array
```

```
int[] b = { 4, 7, 1, 9, 3 };   int[] same = b;
```



Type	Default
int, double	`0`
bool	`false`
string	`null`
char	`'\0'`

read-only

```
foreach (int v in b)
{
    v = 0; // CS1656
}
```

`b[b.Length]` compiles, but crashes: IndexOutOfRangeException.

Linear Search vs Binary Search

Linear search (any array)

```
index ← -1
i ← 0
WHILE i < n AND index = -1
  IF a[i] = target THEN
    index ← i
  END IF
  i ← i + 1
END WHILE
```

Binary search (SORTED array)

```
low ← 0, high ← n - 1
WHILE low ≤ high
  mid ← (low + high) div 2
  IF a[mid] = target THEN
    found at mid, stop
  ELSE IF a[mid] < target
    THEN low ← mid + 1
  ELSE high ← mid - 1
END WHILE
```

n	linear	binary
15	15	4
100	100	7
1 000	1 000	10
10^6	10^6	20

worst-case steps



Exam habit: for a binary search trace, write one row per step: low, high, mid, a[mid], decision. Stop when found or when low > high (not found).

Bubble Sort vs Selection Sort

Bubble sort (early exit)

```
for (int pass = 1; pass <= n - 1; pass++)
{
    bool swapped = false;
    for (int j = 0; j < n - pass; j++)
    {
        if (a[j] > a[j + 1])
        {
            (a[j], a[j + 1]) = (a[j + 1], a[j]);
            swapped = true;
        }
    }
    if (!swapped) { break; }
}
```

Selection sort

```
for (int i = 0; i < n - 1; i++)
{
    int minIndex = i;
    for (int j = i + 1; j < n; j++)
    {
        if (a[j] < a[minIndex])
        {
            minIndex = j;
        }
    }
    (a[i], a[minIndex]) = (a[minIndex], a[i]);
}
```

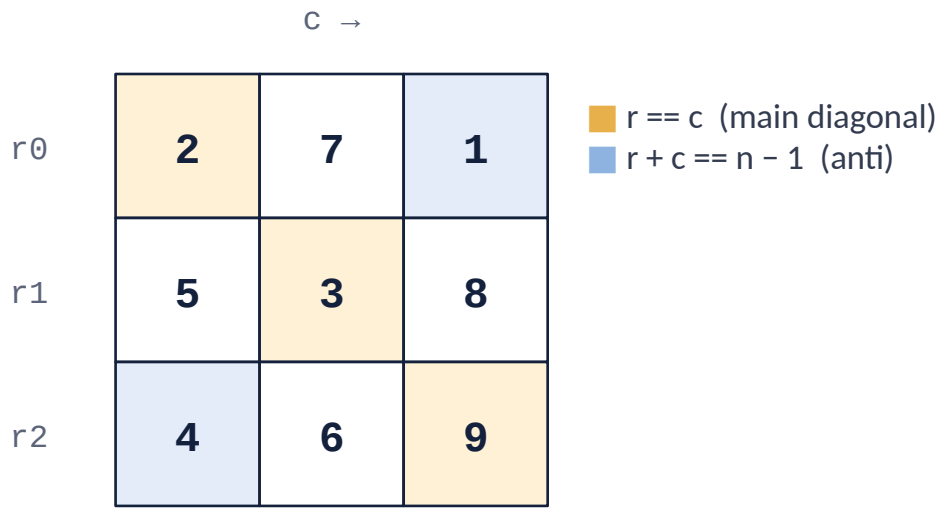
	Bubble	Selection
Idea	swap neighbours	find min, one swap
Comparisons (n = 6)	15 (fewer with early exit)	always 15
Swaps per pass	many	at most 1

Matrices: Rows, Columns and Nested Loops

Row-major traversal

```
int[,] m = { { 2, 7, 1 }, { 5, 3, 8 } };
int rows = m.GetLength(0);      // 2
int cols = m.GetLength(1);      // 3

for (int r = 0; r < rows; r++)  // row sums
{
    int sum = 0;
    for (int c = 0; c < cols; c++)
    {
        sum += m[r, c];
    }
    Console.WriteLine($"row {r}: {sum}");
}
// column sums: swap the two loops
// m.Length == 6 (all elements)
```



Task	Code
element	`m[r, c]`
rows / cols	`m.GetLength(0) / (1)`
transpose	`t[c, r] = m[r, c]`
jagged row	`p[r].Length`

Methods and Parameter Passing

return type · name · parameter list

```
// returns the average of the elements
static double Average(int[] a)
{
    ...
    return (double)sum / a.Length;
}
```

Calls

```
int[] t = { 4, 8 };
double avg = Average(t);          // value
Swap(ref x, ref y);               // ref
bool ok = int.TryParse(s, out int n);
PrintArray(t);                    // void
```

Parameter	Method gets	Caller sees changes?
`int v`	a copy of the value	no
`int[] a`	a copy of the reference	elements: yes · a = new …: no
`ref int v`	the caller's variable	yes
`out int v`	must assign it	yes (that is the purpose)



Prefer returning a result over changing parameters. Use ref only when a method must change two variables (Swap).

return, Scope and Recursion

return

```
static int Sign(int v)
{
    if (v > 0)
    {
        return 1;
    }
    if (v < 0)
    {
        return -1;
    }
    return 0;  // all paths
}
```

Missing last return → CS0161

Scope

```
for (int i = 0; i < 3; i++)
{
    int sq = i * i;
}
// sq, i: CS0103 here

int total = 0;
if (total == 0)
{
    // int total = 1;
    // → CS0136
}
```

Block = lifetime of a local

Recursion

```
static long Fact(int n)
{
    if (n <= 1)
    {
        return 1;   // base
    }
    return n * Fact(n - 1);
}
// Fact(4) → 4·Fact(3)
//  → 4·3·Fact(2) → … 24
```

No base case → stack overflow

Files: Write, Append, Read, Split

Method	Does
`File.WriteAllText(p, s)`	create / overwrite with text
`File.AppendAllText(p, s)`	add at the end
`File.WriteAllLines(p, arr)`	one array element per line
`File.ReadAllLines(p)`	returns string[] (one per line)
`File.ReadAllText(p)`	returns the whole text
`File.Exists(p)`	bool, no exception



No using, no Close: the StreamWriter buffer is not flushed and the file can stay empty (tested: 0 bytes).

Line by line + exceptions

```
try
{
    using (StreamReader r = new StreamReader("in.csv"))
    {
        string line;
        while ((line = r.ReadLine()) != null)
        {
            string[] f = line.Split(',');
            int score = int.Parse(f[1]);
            ...
        }
    }
}
catch (FileNotFoundException)
{
    Console.WriteLine("No such file.");
}
catch (FormatException) { ... }
```

Eight Array Patterns Behind Most Exam Programs

#	Pattern	Core code	Watch out
1	Sum / average	`sum += a[i];  (double)sum / a.Length`	integer division
2	Max with index	`if (a[i] > a[best]) best = i;`	start best = 0 (index), max = a[0]
3	Count if	`if (cond(a[i])) count++;`	count before allocating
4	Linear search	`if (a[i] == x) return i;  …  return -1;`	-1 only after the loop
5	Swap	`int t = a[i]; a[i] = a[j]; a[j] = t;`	three steps, temp first
6	Reverse in place	`for i < n / 2: swap a[i], a[n - 1 - i]`	n / 2, not n
7	Frequency array	`counts[value - offset]++;`	offset: 1 for dice, 'a' for letters
8	Copy / filter	`int[] b = new int[k]; … b[w++] = a[i];`	b = a is NOT a copy



PART 2

Practice I: Read & Trace

Multiple choice · arrays · bubble sort · binary search · matrices · calls · recursion

MULTIPLE CHOICE · 2 min

Quick-Fire Multiple Choice

Q1 double[] t = new double[4]; What does Console.WriteLine(t[4]) do?

- A) prints 0
- B) prints 0.0
- C) prints nothing
- D) throws an exception

Q2 int[,] m = new int[3, 5]; Value of m.GetLength(1)?

- A) 3
- B) 5
- C) 15
- D) 2

Q3 Which call lets a method change the caller's int variable x?

- A) `Change(x)`
- B) `Change(ref x)`
- C) `Change(x + 1)`
- D) `Change((int)x)`

Q4 Return type of File.ReadAllLines(path)?

- A) `string`
- B) `string[]`
- C) `char[]`
- D) `void`

ANSWERS

Quick-Fire Multiple Choice: Answers

Q1 **double[] t = new double[4]; What does Console.WriteLine(t[4]) do?**

- A) prints 0
- B) prints 0.0
- C) prints nothing
- **D) throws an exception**

Valid indices are 0 … 3. It compiles, but throws IndexOutOfRangeException.

Q2 **int[,] m = new int[3, 5]; Value of m.GetLength(1)?**

- A) 3
- **B) 5**
- C) 15
- D) 2

Dimension 1 = columns. GetLength(0) = 3 rows; m.Length = 15.

Q3 **Which call lets a method change the caller's int variable x?**

- A) `Change(x)`
- **B) `Change(ref x)`**
- C) `Change(x + 1)`
- D) `Change((int)x)`

Only ref passes the variable itself (the parameter must be declared ref int).

Q4 **Return type of File.ReadAllLines(path)?**

- A) `string`
- **B) `string[]`**
- C) `char[]`
- D) `void`

One string per line, ready for a for loop and Split.

PREDICT · 4 min

Predict the Output: Six Lines

Program.cs

```
 1  int[] a = { 4, 7, 1, 9, 3 };
 2  int[] b = a;
 3  b[0] = 10;
 4  Console.WriteLine(a[0]);
 5  Console.WriteLine(a[a.Length - 1]);
 6  Console.WriteLine(a[a[2]]);
 7  int sum = 0;
 8  for (int i = 1; i < a.Length; i += 2)
 9  {
10      sum += a[i];
11  }
12  Console.WriteLine(sum);
13  int[] c = (int[])a.Clone();
14  c[1] = 0;
15  Console.WriteLine(string.Join(",", a));
16  Console.WriteLine(a == c);
```

Your answer

1 _____

2 _____

3 _____

4 _____

5 _____

6 _____

ANSWER

Predict the Output: Answer

#	Output	Why
1	`10`	b = a: same array, two names
2	`3`	last index is Length − 1 = 4
3	`7`	a[2] = 1, so a[a[2]] = a[1]
4	`16`	i = 1, 3: 7 + 9
5	`10,7,1,9,3`	the clone c is a different array
6	`False`	== compares references, not contents



Copy ≠ second name: int[] b = a; shares the array. (int[])a.Clone() or new + loop makes a real copy.

TRACE · 5 min

Trace the Bubble Sort, Pass by Pass

```
int[] a = { 4, 2, 7, 1, 9, 5 };
int n = a.Length;
for (int pass = 1; pass <= n - 1; pass++)
{
    bool swapped = false;
    for (int j = 0; j < n - pass; j++)
    {
        if (a[j] > a[j + 1])
        {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
            swapped = true;
        }
    }
    if (!swapped)
    {
        break;
    }
}
```

Pass	Array after the pass	Swaps
start	4 2 7 1 9 5	–
1		
2		
3		
4		
5		

? After which pass does the loop stop? Total comparisons? Total swaps?

ANSWER

Bubble Sort Trace: Answer

Pass	Array after the pass	Comparisons	Swaps
start	4 2 7 1 9 5	–	–
1	2 4 1 7 5 9	5	3
2	2 1 4 5 7 9	4	2
3	1 2 4 5 7 9	3	1
4	1 2 4 5 7 9	2	0 → break
Total	*pass 5 never runs*	**14**	**6**

15_01_PredictChecker

```
Pass 1: 2 4 1 7 5 9
Pass 2: 2 1 4 5 7 9
Pass 3: 1 2 4 5 7 9
Pass 4: 1 2 4 5 7 9
```

After pass k the k largest values are final.

Pass 3 already sorts the array; pass 4 is needed to DETECT it (no swap).

TRACE · 5 min

Trace the Binary Search

a	3	8	11	15	19	24	30	36	41	47	52
	0	1	2	3	4	5	6	7	8	9	10

```
int low = 0, high = a.Length - 1, index = -1;
while (low <= high && index == -1)
{
    int mid = (low + high) / 2;
    if (a[mid] == target)
    {
        index = mid;
    }
    else if (a[mid] < target)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
```

target	step	low	high	mid	a[mid]
47	1				
	2				
	3				
12	1				
	2				
	3				

Result for each target? How many steps?

ANSWER

Binary Search Trace: Answer

a	3	8	11	15	19	24	30	36	41	47	52
	0	1	2	3	4	5	6	7	8	9	10

target	step	low	high	mid	a[mid]	decision
47	1	0	10	5	24	24 < 47 → low = 6
	2	6	10	8	41	41 < 47 → low = 9
	3	9	10	9	47	**found: index 9**
12	1	0	10	5	24	24 > 12 → high = 4
	2	0	4	2	11	11 < 12 → low = 3
	3	3	4	3	15	15 > 12 → high = 2
		3	2			**low > high: -1**



3 steps each

Linear search: 10 checks for 47, 11 for 12. Binary search halves the range at every step, but needs SORTED data.

PREDICT · 4 min

Predict the Output: Nested Loops on a Matrix

(a)

```
int[,] m = { { 2, 7, 1 },
             { 5, 3, 8 },
             { 4, 6, 9 } };
int n = m.GetLength(0);
for (int r = 0; r < n; r++)
{
    for (int c = 0; c < n; c++)
    {
        if (r > c)
        {
            Console.Write("  .");
        }
        else
        {
            Console.Write($"{m[r, c],3}");
        }
    }
    Console.WriteLine();
}
```

(b) continues (a)

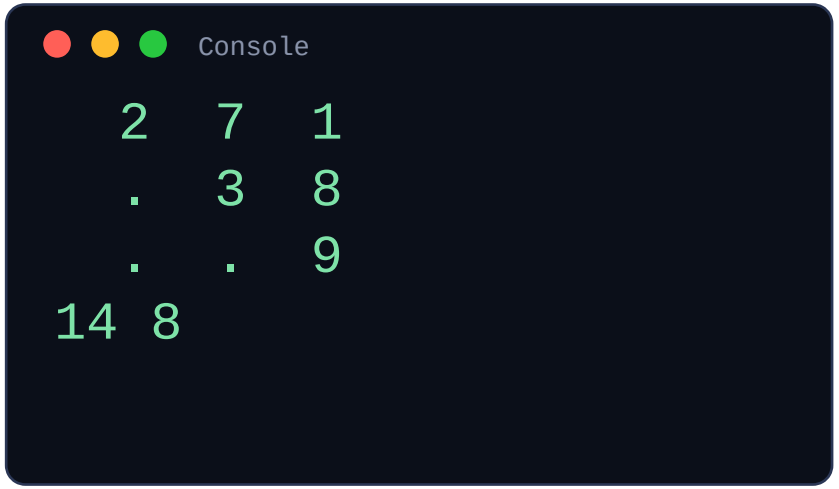
```
int diag = 0, anti = 0;
for (int i = 0; i < n; i++)
{
    diag += m[i, i];
    anti += m[i, n - 1 - i];
}
Console.WriteLine($"{diag} {anti}");
```

(a) Which cells print " ." and which print a value?

(b) List the (r, c) pairs added to diag and to anti, then the output.

ANSWER

Nested Loops on a Matrix: Answer



2	**7**	**1**
5	**3**	**8**
4	6	**9**

grey: r > c (printed as " .")

i	m[i, i]	m[i, n−1−i]
0	2	1
1	3	3
2	9	4
sum	**14**	**8**

Index rules for an n × n matrix: main diagonal r == c · above it r < c · below it r > c · anti-diagonal r + c == n − 1. The center (1,1) belongs to both diagonals.

PREDICT · 5 min

Method Calls: Value, ref and Array Parameters

Top-level statements

```
int x = 5, y = 5;
int[] arr = { 5, 5 };
Bump(x);
BumpRef(ref y);
BumpArray(arr);
Replace(arr);
Console.WriteLine($"{x} {y} {arr[0]} {arr[1]}");
int z = Twice(x) + Twice(Twice(1));
Console.WriteLine(z);
```

```
static void Replace(int[] a)
{
    a = new int[] { 0, 0 };
    a[1] = 99;
}
static int Twice(int v)
{
    return 2 * v;
}
```

```
static void Bump(int v)
{
    v++;
}
static void BumpRef(ref int v)
{
    v++;
}
static void BumpArray(int[] a)
{
    a[0]++;
}
```

? Write the two output lines. Which calls change something the caller can see?

ANSWER

Value, ref and Array Parameters: Answer

Console

```
5 6 6 5
14
```

Rule

The argument is always COPIED. For an array the copy is a reference: same elements, but a new array inside the method is invisible outside.

Call	Method receives	Effect for the caller
`Bump(x)`	copy of 5	none: x = 5
`BumpRef(ref y)`	y itself	y = 6
`BumpArray(arr)`	copy of the reference	arr[0] = 6
`Replace(arr)`	copy of the reference	none: a points to a new array
`Twice(Twice(1))`	inner call first: 2	4; z = 10 + 4 = 14

To give a result back: return it (Twice), or use ref/out. Changing a value parameter is lost.

TRACE · 5 min

Trace the Recursion

(a) Console.WriteLine(Mystery(6));

```
static int Mystery(int n)
{
    if (n == 0)
    {
        return 0;
    }
    return n % 2 + 10 * Mystery(n / 2);
}
```

(a) Mystery(6) = ? How many calls? What does Mystery compute?

(b) Exact output of Countdown(3)?

(b) Countdown(3);

```
static void Countdown(int n)
{
    if (n == 0)
    {
        return;
    }
    Console.Write(n + " ");
    Countdown(n - 1);
    Console.Write(n + " ");
}
```



One box per call; returns go back up.

ANSWER

Recursion Trace: Answer

Mystery(6) 0 + 10 · Mystery(3) → **110**

Mystery(3) 1 + 10 · Mystery(1) → **11**

Mystery(1) 1 + 10 · Mystery(0) → **1**

Mystery(0) base case → **0**

(a)

```
110
```

(b)

```
3 2 1 1 2 3
```

No base case (or no progress) → StackOverflowException.

4 calls (6, 3, 1, 0). Mystery writes n in binary: 6 → 110. In (b) the second Write runs on the way back up.



PART 3

Practice II: Debug & Write

Scope errors · find the bugs · write methods · a file program · strategy

FIND ERRORS · 4 min

Spot the Compile Errors

```
 1  int total = 0;
 2  for (int i = 1; i <= 3; i++)
 3  {
 4      int square = i * i;
 5      total += square;
 6  }
 7  Console.WriteLine(square);
 8  if (total > 10)
 9  {
10      int total = 0;
11      string msg = "big";
12  }
13  else
14  {
15      string msg = "small";
16  }
17  int count;
18  Console.WriteLine(count + total);
19  Console.WriteLine(Half(total));
```

```
Line 20 is empty; the method starts at line 21
21  static int Half(int n)
22  {
23      if (n >= 0)
24      {
25          return n / 2;
26      }
27  }
```



4 errors

Which lines? Why?



Trap

Is msg declared twice illegally?

ANSWER

Spot the Compile Errors: Answer

Line	Code	Error	Reason
7	`WriteLine(square)`	`CS0103`	square lives only inside the loop block
10	`int total = 0;`	`CS0136`	nested block reuses a name of the enclosing scope
18	`count + total`	`CS0165`	count is declared but never assigned
21	`static int Half(…)`	`CS0161`	no return when n < 0: not all paths return
11, 15	`string msg`	**OK**	sibling blocks: two different variables (warning only)

Rule: a variable exists from its declaration to the } of its block. An inner block may not reuse an outer name; two sibling blocks may.

FIND BUGS · 5 min

Find the Bugs: Winter Temperatures

```
1  int[] temps = new int[5];
2  for (int i = 1; i <= temps.Length; i++)
3  {
4      Console.Write($"Day {i}: ");
5      temps[i] = int.Parse(Console.ReadLine());
6  }
7  Console.WriteLine($"Average: {Average(temps):F2}");
8  Console.WriteLine($"Highest: {Highest(temps)}");
9
10 static double Average(int[] a)
11 {
12     int sum = 0;
13     foreach (int t in a)
14     {
15         sum += t;
16     }
17     return sum / a.Length;
18 }
```

```
20 static int Highest(int[] a)
21 {
22     int max = 0;
23     for (int i = 0; i < a.Length; i++)
24     {
25         if (a[i] > max)
26         {
27             max = i;
28         }
29     }
30     return max;
31 }
```

4 wrong lines

Test with -3, -7, -2, -1, -4.

ANSWER

Find the Bugs: Answer

Line	Bug	Fix
2	i = 1 … Length: skips a[0], a[5] crashes	`for (int i = 0; i < temps.Length; i++)`
17	sum / a.Length is integer division	`return (double)sum / a.Length;`
22	max = 0 is wrong for all-negative data	`int max = a[0];`
27	stores the index, not the value	`max = a[i];`



Test with negative data: only then do "max = 0" and the integer division show. Prompt in the fix: Day {i + 1}.

Buggy version, real run

```
Day 5: -4
Unhandled exception.
System.IndexOutOf
RangeException
```

15_02_TempStatsFixed

```
Average: -3.40
Highest: -1
```

WRITE CODE · 8 min

Write the Methods: Weekly Step Counter

Given: top-level statements

```
string[] days = { "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun" };
int[] steps = ReadSteps(days);

double average = Average(steps);
int best = IndexOfMax(steps);
int above = CountAbove(steps, average);
Console.WriteLine($"Average: {average:F1} steps");
Console.WriteLine($"Best day: {days[best]} ({steps[best]} steps)");
Console.WriteLine($"Days above average: {above}");
```

Write these three methods

1. Average(steps): the mean as a double
2. IndexOfMax(steps): index of the first largest value
3. CountAbove(steps, limit): how many are > limit

Required sample run

```
Mon: 6500
Tue: 8200
Wed: abc
  Please enter a whole
  number >= 0.
Wed: 7400
Thu: 12050
Fri: 9900
Sat: 3100
Sun: 10400
Average: 8221.4 steps
Best day: Thu (12050 steps)
Days above average: 3
```

ANSWER

Weekly Step Counter: the Three Methods

```
static double Average(int[] a)
{
    int sum = 0;
    foreach (int x in a)
    {
        sum += x;
    }
    return (double)sum / a.Length;
}
static int IndexOfMax(int[] a)
{
    int best = 0;
    for (int i = 1; i < a.Length; i++)
    {
        if (a[i] > a[best])
        {
            best = i;
        }
    }
    return best;
}
```

```
static int CountAbove(int[] a, double limit)
{
    int count = 0;
    foreach (int x in a)
    {
        if (x > limit)
        {
            count++;
        }
    }
    return count;
}
```



Mon … Sun = 57 550 steps, average 8221.4; above it: Thu, Fri, Sun → 3.

WRITE CODE · 10 min

Write the Program: CSV Grades to a Report File

Specification

1. Call the given CreateSampleFile("grades.csv").
2. Read grades.csv with a StreamReader in using; skip the header.
3. Split each line at commas. Not 3 fields, or not a number: print "Skipped (...): line".
4. Total = 0.4 · midterm + 0.6 · final. Write name, total (F2) and PASS/FAIL (≥ 50) to report.txt.
5. Print the count, the class average and report.txt. Catch FileNotFoundException.

grades.csv (created by the program)

```
name,midterm,final
Ayşe,72,85
Mehmet,45,58
Zeynep,90,94
Can,60,70
Deniz,80
Elif,38,51
```

? Two lines are bad. Which ones, and which check catches each?

ANSWER

CSV Grades: Structure and Helper Methods

```
int count = 0;
double sum = 0;
try
{
    using (StreamReader reader = new StreamReader(InputFile))
    using (StreamWriter writer = new StreamWriter(ReportFile))
    {
        reader.ReadLine();
        string line;
        while ((line = reader.ReadLine()) != null)
        {
            // loop body: next slide
        }
    }
}
catch (FileNotFoundException)
{
    Console.WriteLine($"File not found: {InputFile}");
}
PrintSummary(count, sum, ReportFile);
```

```
static double Total(string mid, string fin)
{
    return 0.4 * int.Parse(mid)
         + 0.6 * int.Parse(fin);
}
```

```
static void PrintSummary(int count,
    double sum, string path)
{
    if (count == 0)
    {
        Console.WriteLine("No valid lines.");
        return;
    }
    double avg = sum / count;
    Console.WriteLine($"Students: {count}");
    Console.WriteLine($"Average: {avg:F2}");
    Console.WriteLine($"--- {path} ---");
    Console.Write(File.ReadAllText(path));
}
```

ANSWER

CSV Grades: the Loop Body and the Real Run

```
string[] parts = line.Split(',');
if (parts.Length != 3)
{
    Console.WriteLine($"Skipped (3 fields expected): {line}");
    continue;
}
try
{
    double total = Total(parts[1], parts[2]);
    string result = total >= 50 ? "PASS" : "FAIL";
    writer.WriteLine($"{parts[0],-8}{total,7:F2}  {result}");
    count++;
    sum += total;
}
catch (FormatException)
{
    Console.WriteLine($"Skipped (not a number): {line}");
}
```

Real run (long lines wrapped)

```
Skipped (not a number):
  Can,6O,70
Skipped (3 fields expected):
  Deniz,80
Students: 4
Average: 67.70
--- report.txt ---
Ayşe      79.80  PASS
Mehmet    52.80  PASS
Zeynep    92.40  PASS
Elif      45.80  FAIL
```



continue skips the rest of this pass; the catch lets the loop go on.

Ten Mistakes That Cost Points in the Second Half

#	Wrong	Right	Why
1	`for (i = 0; i <= a.Length; i++)`	`i < a.Length`	IndexOutOfRangeException
2	`int max = 0;`	`int max = a[0];`	fails for negative data
3	`int[] copy = a;`	`(int[])a.Clone()  or new + loop`	same array, two names
4	`m.Length as number of rows`	`m.GetLength(0)`	Length = all elements
5	`m[r][c]  on int[,]`	`m[r, c]`	CS0022
6	`else { return false; } in a search`	`return false after the loop`	decides after 1 element
7	`value method without final return`	`every path returns`	CS0161
8	`void Bump(int v) to change x`	`return the value, or ref`	v is only a copy
9	`new StreamWriter(…) without using`	`using (StreamWriter w = …)`	file may stay empty
10	`int.Parse(f[1]) with no try/catch`	`catch (FormatException)`	one bad line stops all

>_ EXAM STRATEGY

A 120-Minute Plan and a Method per Question Type



Read all: 5 min

Arrays & matrices

Draw the boxes with their indices. Update one cell at a time. Write r and c next to each value.

Calls & recursion

One box per call: parameters, pending expression, returned value. Copy or ref?

Bugs & scope

Check bounds (< Length), types, returns, braces/blocks, Parse, using.

Write a program

Method headers first, then main, then bodies. Name the pattern. Prompts and output format last.

Never leave a question blank: a correct header, pseudocode or a partial trace usually earns partial credit.

>_ BEFORE THE EXAM

Your Study Plan for the Final

Day 1

Cheat sheet + Weeks 9–11 slides. Write your own one-page summary.

Day 2

Weeks 12–14: methods, scope, recursion, files. Redo Q9–Q14.

Day 3

Practice final: 120 minutes, on paper, with a timer.

Day 4

Check it with the code folder. List every mistake and its cause.

Day 5

Problem set P1–P14; type the programs and compare.

Code folder: 15 tested projects (15_01 … 15_15): every programming question of this deck, the practice final and the problem set.

>_ SUMMARY

Key Takeaways

01 The final is cumulative (Weeks 1–14); most points come from arrays, matrices, methods and files.

02 Arrays: indices 0 … Length − 1; b = a is not a copy; max starts with a[0].

03 Trace sorts and searches row by row: pass, low/high/mid, swaps, comparisons.

04 Parameters are copies: a value never changes the caller; ref does; arrays share elements.

05 Every path returns; scope ends at the }; recursion needs a base case.

06 Files: using, ReadLine() until null, Split, Parse inside try/catch.

NEXT WEEK

Final exam: date, room and duration in class



MCMP113
Course materials